

Exercício Prático — Testes Manuais de Frontend Web

Sistema: Banking Frontend Training System

Nível: QA Entry Level

Data: Fevereiro 2026

Objetivo do Exercício

Neste exercício, você irá praticar as atividades essenciais de um QA:

1. Elaborar **10 casos de teste** para o frontend web
2. Criar um **plano de teste** documentando sua estratégia
3. Executar os testes e criar um **relatório de execução**
4. Registrar **bugs encontrados** seguindo boas práticas
5. Calcular a **cobertura de testes** por requisitos

Este exercício simula um cenário real onde você recebe uma especificação de frontend e precisa garantir sua qualidade através de testes manuais.

Como Subir o Sistema

Pré-requisitos

Antes de começar, certifique-se de ter instalado:

- **Node.js** versão 18 ou superior ([Download aqui](#))
- Um **navegador web** (Chrome, Firefox ou Edge)

Passo a Passo

1. Instalar as dependências do backend

Abra o terminal na pasta do projeto e execute:

```
cd backend  
npm install
```

Aguarde a instalação das dependências.

2. Iniciar o servidor da API

Ainda no terminal, dentro da pasta backend, execute:

```
npm run dev
```

Você verá uma mensagem similar a:

Servidor rodando em <http://localhost:3000>
Swagger disponível em <http://localhost:3000/api-docs>

Pronto! A API está rodando.

3. Abrir o Frontend Web

O frontend não precisa de servidor — basta abrir o arquivo HTML no navegador:

- Navegue até a pasta frontend-web
- Abra o arquivo index.html diretamente no navegador (clique duplo ou arraste para o navegador)

Dica: Use a extensão **Live Server** no VS Code para recarregamento automático.

4. Verificar se está funcionando

- Abra as **DevTools** do navegador (pressione F12)
- Vá até a aba **Console** — não deve haver erros
- Tente criar uma conta de teste para validar que frontend e API estão se comunicando

Especificação de Requisitos — Frontend Web

Visão Geral

O **Banking Frontend** é uma interface web luxuosa que permite gerenciar contas bancárias através de uma API REST. A interface possui:

- Cards para diferentes operações bancárias
- Tabela que lista todas as contas
- Feedback visual para operações (sucessos e erros)
- Design responsivo e elegante

Requisitos Funcionais do Frontend

RF-FE001: Criar Nova Conta

Descrição: O usuário deve conseguir criar uma nova conta bancária através do formulário.

Localização: Card "Open New Account"

Elementos da Interface:

- Campo de texto: **Full Name** (obrigatório)
- Campo de email: **Email Address** (obrigatório)
- Campo de texto: **Tax ID (CPF)** (obrigatório)
- Campo numérico: **Initial Deposit** (obrigatório, deve ser ≥ 0)
- Botão: **Create Account**

Comportamento Esperado:

1. Ao clicar em "Create Account", o sistema envia os dados para a API
2. Se sucesso (201):
 - Exibe resposta em JSON verde abaixo do botão
 - Limpa todos os campos do formulário
 - Atualiza automaticamente a tabela de contas
 - Mostra toast de sucesso no canto inferior direito
3. Se erro (400/404):
 - Exibe resposta em JSON vermelho
 - Mantém os dados no formulário
 - Mostra mensagem de erro clara

Regras de Validação:

- Todos os campos são obrigatórios
- Initial Deposit não pode ser negativo
- CPF deve ser único (não pode duplicar)
- Email deve ter formato válido

RF-FE002: Listar Todas as Contas

Descrição: O sistema deve exibir uma tabela com todas as contas cadastradas.

Localização: Seção "All Accounts" (parte inferior da página)

Elementos da Interface:

- Tabela com colunas: Account ID, Name, Email, Tax ID, Balance
- Botão: **Refresh Accounts**

Comportamento Esperado:

1. Ao carregar a página, a tabela deve ser preenchida automaticamente
 2. Account ID exibido de forma abreviada (primeiros 8 caracteres)
 3. Balance formatado com 2 casas decimais
 4. Se não houver contas, exibir mensagem: "No accounts yet. Create your first account above."
 5. Ao clicar em "Refresh Accounts", a tabela é atualizada com dados mais recentes
 6. Efeito hover nas linhas da tabela (fundo muda de cor)
-

RF-FE003: Atualizar Dados da Conta

Descrição: O usuário deve conseguir atualizar o nome e email de uma conta existente.

Localização: Card "Update Account"

Elementos da Interface:

- Campo de texto: **Account ID** (obrigatório)
- Campo de texto: **New Name** (opcional)
- Campo de email: **New Email** (opcional)
- Botão: **Update**

Comportamento Esperado:

1. Usuário informa o ID da conta que deseja atualizar
 2. Preenche novo nome e/ou novo email
 3. Ao clicar em "Update":
 - Se sucesso (200): exibe JSON verde, atualiza tabela, toast de sucesso
 - Se erro (404): exibe JSON vermelho com mensagem "Conta não encontrada"
 4. Não é possível alterar CPF ou Balance através deste formulário
-

RF-FE004: Deletar Conta

Descrição: O usuário deve conseguir remover uma conta do sistema.

Localização: Card "Update Account"

Elementos da Interface:

- Campo de texto: **Account ID** (compartilhado com Update)
- Botão: **Delete Account** (vermelho/secundário)

Comportamento Esperado:

1. Usuário informa o ID da conta
2. Ao clicar em "Delete Account":

- Exibe confirmação: "Are you sure you want to delete this account?"
 - Se confirmar:
 - Se sucesso (200): exibe JSON verde, atualiza tabela, limpa campo ID, toast de sucesso
 - Se erro (404): exibe JSON vermelho
 - Se cancelar: nada acontece
-

RF-FE005: Realizar Depósito

Descrição: O usuário deve conseguir adicionar dinheiro a uma conta existente.

Localização: Card "Deposit Funds"

Elementos da Interface:

- Campo de texto: **Account ID** (obrigatório)
- Campo numérico: **Amount** (obrigatório, deve ser > 0)
- Botão: **Deposit**

Comportamento Esperado:

1. Usuário informa ID da conta e valor a depositar
2. Ao clicar em "Deposit":
 - Se sucesso (200):
 - Exibe JSON verde com novo saldo
 - Atualiza tabela automaticamente
 - Toast de sucesso
 - Limpa os campos
 - Se erro (400): exibe JSON vermelho (ex: "amount deve ser maior que 0")
 - Se erro (404): exibe JSON vermelho (conta não encontrada)

Regras de Validação:

- Amount deve ser maior que 0 (não aceita zero ou negativos)
 - Conta deve existir
-

RF-FE006: Realizar Transferência

Descrição: O usuário deve conseguir transferir dinheiro entre duas contas.

Localização: Card "Transfer Funds"

Elementos da Interface:

- Campo de texto: **From Account ID** (obrigatório)
- Campo de texto: **To Account ID** (obrigatório)
- Campo numérico: **Amount** (obrigatório, deve ser > 0)
- Botão: **Transfer**

Comportamento Esperado:

1. Usuário informa ID da conta origem, ID da conta destino e valor
2. Ao clicar em "Transfer":
 - Se sucesso (200):
 - Exibe JSON verde com saldos atualizados de ambas as contas
 - Atualiza tabela automaticamente
 - Toast de sucesso
 - Limpa os campos
 - Se erro (400): exibe JSON vermelho (ex: "Saldo insuficiente" ou "Origem e destino não podem ser iguais")
 - Se erro (404): exibe JSON vermelho (conta origem ou destino não encontrada)

Regras de Validação:

- Amount deve ser maior que 0
- Conta origem deve existir
- Conta destino deve existir
- Conta origem ≠ conta destino
- Conta origem deve ter saldo suficiente

RF-FE007: Feedback Visual de Operações

Descrição: O sistema deve fornecer feedback claro sobre o resultado das operações.

Elementos da Interface:

- Blocos de resposta JSON abaixo de cada botão de ação
- Toast notifications no canto inferior direito
- Estados visuais dos botões (hover, loading)

Comportamento Esperado:

1. **Resposta JSON:**
 - Fundo verde claro para sucessos (200, 201)
 - Fundo vermelho claro para erros (400, 404)
 - JSON formatado e legível

- Aparece apenas após uma ação
 - 2. **Toast Notifications:**
 - Aparece no canto inferior direito
 - Desaparece automaticamente após 3 segundos
 - Fundo dourado com texto escuro
 - Animação de entrada suave
 - 3. **Loading States:**
 - Botões mostram estado de carregamento durante requisição
 - Previne múltiplos cliques
-

RF-FE008: Design Responsivo

Descrição: A interface deve adaptar-se a diferentes tamanhos de tela.

Comportamento Esperado:

1. Em telas grandes (> 1200px):
 - Cards dispostos em grid de 2 colunas
 - Tabela com todas as colunas visíveis
 2. Em telas médias (768px - 1200px):
 - Cards em coluna única
 - Tabela mantém todas as colunas
 3. Em telas pequenas (< 768px):
 - Header em coluna única
 - Navegação empilhada
 - Cards em coluna única
 - Tabela pode ter scroll horizontal se necessário
-

Parte 1: Elaborar 10 Casos de Teste

O que é um Caso de Teste?

Um **caso de teste** é um documento que descreve:

- **O QUE** testar (funcionalidade específica)
- **COMO** testar (passos detalhados)
- **RESULTADO ESPERADO** (o que deve acontecer)

Template de Caso de Teste

Use o seguinte formato para cada caso de teste:

Campo	Descrição
ID	Identificador único (ex: CT-FE-001)
Título	Resumo claro do que será testado
Pré-condições	Estado do sistema antes do teste
Passos	Sequência de ações a executar (numerada)
Resultado Esperado	O que deve acontecer (detalhado)
Requisito Relacionado	RF-FE-XXX

Exemplo Completo de Caso de Teste

ID: CT-FE-001

Título: Criar conta com dados válidos

Pré-condições:

- Backend rodando em <http://localhost:3000>
- Frontend aberto no navegador
- Nenhuma conta com CPF "123.456.789-00" cadastrada previamente

Passos:

1. Localizar o card "Open New Account"
2. Preencher o campo "Full Name" com "Maria Santos"
3. Preencher o campo "Email Address" com "maria@email.com"
4. Preencher o campo "Tax ID (CPF)" com "123.456.789-00"
5. Preencher o campo "Initial Deposit" com "1000"
6. Clicar no botão "Create Account"
7. Aguardar a resposta (1-2 segundos)
8. Observar o bloco de resposta JSON abaixo do botão
9. Observar a tabela "All Accounts" na parte inferior
10. Observar o toast notification no canto inferior direito

Resultado Esperado:

- Bloco de resposta JSON aparece com **fundo verde**
- JSON contém:
 - id (string UUID)

- name: "Maria Santos"
- email: "maria@email.com"
- cpf: "123.456.789-00"
- balance: 1000
- createdAt (data/hora atual)
- **Todos os campos do formulário são limpos** (voltam vazios)
- **Tabela "All Accounts" é atualizada automaticamente** com a nova conta
- **Toast aparece** no canto inferior direito com mensagem "Account created successfully!"
- Toast desaparece após 3 segundos

Requisito Relacionado: RF-FE001

Exemplo de Caso de Teste Negativo

ID: CT-FE-002

Título: Tentar criar conta com CPF duplicado

Pré-condições:

- Backend rodando
- Frontend aberto no navegador
- Já existe uma conta cadastrada com CPF "111.222.333-44"

Passos:

1. Localizar o card "Open New Account"
2. Preencher o campo "Full Name" com "João Silva"
3. Preencher o campo "Email Address" com "joao@email.com"
4. Preencher o campo "Tax ID (CPF)" com "111.222.333-44" (CPF já existente)
5. Preencher o campo "Initial Deposit" com "500"
6. Clicar no botão "Create Account"
7. Aguardar a resposta

Resultado Esperado:

- Bloco de resposta JSON aparece com **fundo vermelho**
- JSON contém mensagem de erro indicando CPF duplicado
- **Campos do formulário NÃO são limpos** (dados permanecem para correção)
- Tabela "All Accounts" **não é alterada**
- **Nenhum toast de sucesso** aparece

Seu Desafio

Elabore **10 casos de teste** seguindo o template acima.

Sugestão de cobertura:

Criação de Conta (RF-FE001):

- 1 caso com dados válidos
- 1 caso com dados inválidos (CPF duplicado, campo vazio, ou saldo negativo)

Listagem (RF-FE002):

- 1 caso testando refresh da tabela

Atualização (RF-FE003):

- 1 caso atualizando conta existente

Deleção (RF-FE004):

- 1 caso deletando conta existente

Depósito (RF-FE005):

- 1 caso com valor válido
- 1 caso com valor inválido (zero ou negativo)

Transferência (RF-FE006):

- 1 caso com transferência válida
- 1 caso com saldo insuficiente ou contas iguais

UI/UX (RF-FE007 e RF-FE008):

- 1 caso testando feedback visual (cores, toast)

Dica: Teste tanto os **caminhos felizes** (tudo funciona) quanto os **caminhos de erro** (validações, dados inválidos).

Parte 2: Elaborar o Plano de Teste

O que é um Plano de Teste?

Um **plano de teste** é um documento estratégico que define:

- **Escopo** — o que será testado (e o que não será)
- **Abordagem** — como os testes serão executados

- **Recursos** — ferramentas e ambiente necessários
- **Cronograma** — quando os testes acontecerão
- **Critérios de aceite** — quando considerar "pronto"

Template de Plano de Teste

Use a seguinte estrutura:

1. Identificação

- **Projeto:** Banking Frontend Training System
- **Versão:** 1.0
- **Responsável:** [Seu nome]
- **Data:** [Data atual]

2. Escopo

O que será testado:

- Criar conta (RF-FE001)
- Listar contas (RF-FE002)
- Atualizar conta (RF-FE003)
- Deletar conta (RF-FE004)
- Depósito (RF-FE005)
- Transferência (RF-FE006)
- Feedback visual (RF-FE007)
- Design responsivo (RF-FE008)

O que NÃO será testado:

- API de forma isolada (já possui testes próprios)
- Performance e carga
- Segurança e autenticação
- Compatibilidade com navegadores antigos (apenas Chrome/Firefox/Edge modernos)
- Frontend mobile (fora do escopo deste exercício)

3. Abordagem

- **Tipo de teste:** Testes funcionais manuais de interface web
- **Ferramentas:**
 - Navegador Chrome (ou Firefox/Edge)
 - DevTools do navegador (F12)

- Backend API rodando localmente
- **Ambiente:** Local (<http://localhost:3000> + index.html)
- **Dados de teste:** Massas de dados criadas durante a execução
- **Técnica:** Testes exploratórios + testes baseados em casos de teste documentados

4. Recursos Necessários

- Node.js instalado
- Backend rodando (npm run dev)
- Navegador web moderno
- Arquivo index.html do frontend
- Documentação dos requisitos

5. Cronograma

Atividade	Duração Estimada
Elaboração dos casos de teste	1 hora
Configuração do ambiente	15 minutos
Execução dos testes	1 hora
Registro de bugs	30 minutos
Relatório final	30 minutos
TOTAL	3 horas 15 minutos

6. Critérios de Aceite

Critérios de sucesso:

- Todos os requisitos funcionais (RF-FE001 a RF-FE008) possuem pelo menos 1 caso de teste
- Casos de teste cobrem caminhos felizes e de erro
- Todos os casos foram executados
- Bugs críticos foram registrados
- Cobertura de requisitos $\geq 80\%$

Critério de bloqueio:

- Se o backend não subir ou a comunicação frontend-API estiver quebrada, os testes são suspensos até correção

Exemplo Simplificado de Plano de Teste

1. Identificação

- Projeto: Banking Frontend
- Responsável: Ana Silva
- Data: 23/02/2026

2. Escopo

- Testarei: Todos os 8 requisitos do frontend
- Não testarei: API isolada, performance

3. Abordagem

- Testes manuais no Chrome usando DevTools

4. Recursos

- Chrome, backend rodando, index.html

5. Cronograma

- Total: 3h15min

6. Critérios

- Sucesso: 80% cobertura, bugs registrados
- Bloqueio: Backend não funciona

Seu Desafio

Preencha o plano de teste acima, adaptando:

- Seu nome como responsável
- Data atual
- Estimativas de tempo realistas para você
- Qualquer observação adicional relevante

Parte 3: Executar os Testes e Criar Relatório de Execução

O que é um Relatório de Execução?

É um documento que registra o **resultado real** da execução dos casos de teste, incluindo:

- **Status** (passou, falhou, bloqueado)
- **Evidências** (prints, logs do console)

- **Observações** sobre comportamentos inesperados

Template de Relatório de Execução

ID	Título	Status	Observações	Evidência
CT-FE-001	Criar conta válida	Passou	Conta criada, tabela atualizada	Print tela
CT-FE-002	CPF duplicado	Falhou	Aceitou CPF duplicado	Bug #001
CT-FE-003	Listar contas	Passou	Tabela exibiu 2 contas	-

Legenda de Status:

- **Passou** — Resultado obtido = Resultado esperado
- **Falhou** — Resultado obtido $\neq$ Resultado esperado (bug encontrado)
- **Bloqueado** — Não foi possível executar (dependência quebrada)
- **Não Executado** — Ainda não foi testado

Exemplo Detalhado de Registro de Execução

CT-FE-001: Criar conta com dados válidos

Executado em: 23/02/2026 às 22:45

Executado por: Ana Silva

Navegador: Chrome 120

Passos executados:

1. Localizei o card "Open New Account"
2. Preenchi "Full Name" com "Maria Santos"
3. Preenchi "Email Address" com "maria@email.com"
4. Preenchi "Tax ID (CPF)" com "123.456.789-00"
5. Preenchi "Initial Deposit" com "1000"
6. Cliquei no botão "Create Account"
7. Aguardei 1 segundo
8. Observei o bloco de resposta JSON
9. Observei a tabela "All Accounts"
10. Observei o toast notification

Resultado obtido:

- Bloco JSON apareceu com fundo verde
- JSON continha todos os campos esperados

- Campos do formulário foram limpos
- Tabela atualizou automaticamente
- Toast "Account created successfully!" apareceu e desapareceu

Comparação:

- Status esperado: Sucesso com fundo verde
- Campos limpos: Sim
- Tabela atualizada: Sim
- Toast exibido: Sim

Conclusão: PASSOU**Evidências:**

- Screenshot do JSON de resposta
 - Screenshot da tabela atualizada
 - Console do DevTools sem erros
-

Exemplo de Execução com Falha

CT-FE-002: Tentar criar conta com CPF duplicado

Executado em: 23/02/2026 às 22:50

Executado por: Ana Silva

Passos executados:

1. Criei uma primeira conta com CPF "111.222.333-44"
2. Tentei criar segunda conta com mesmo CPF
3. Observei a resposta

Resultado obtido:

- Bloco JSON apareceu com fundo verde (ERRADO!)
- Status HTTP: 201 Created (deveria ser 400)
- Segunda conta foi criada com CPF duplicado
- Tabela mostra 2 contas com mesmo CPF

Resultado esperado:

- Bloco JSON com fundo vermelho
- Status HTTP: 400 Bad Request
- Mensagem de erro clara
- Conta não criada

Conclusão: FALHOU

Bug registrado: BUG-FE-001

Evidências:

- Screenshot mostrando 2 contas com mesmo CPF
 - Console do DevTools com resposta 201
 - Network tab mostrando request/response
-

Seu Desafio

Execute cada um dos **10 casos de teste** que você criou e preencha o relatório de execução com:

- Data e hora da execução
- Navegador utilizado
- Status (passou/falhou/bloqueado)
- Observações detalhadas sobre o comportamento
- Se falhou, referencie o bug registrado
- (Opcional) Anexe prints de tela como evidência

Dica: Use as **DevTools** (F12) para:

- Ver requisições na aba **Network**
 - Ver erros JavaScript na aba **Console**
 - Inspecionar elementos na aba **Elements**
 - Fazer screenshots de evidências
-

Parte 4: Registrar Bugs Encontrados

O que é um Registro de Bug?

É um documento que descreve um **comportamento incorreto** do sistema de forma clara e completa para que o desenvolvedor possa reproduzir e corrigir.

Template de Registro de Bug

Campo	Descrição
ID	Identificador único (ex: BUG-FE-001)
Título	Resumo do problema (máximo 80 caracteres)

Severidade	Crítica / Alta / Média / Baixa
Prioridade	Alta / Média / Baixa
Ambiente	Local (navegador, SO, versão)
Pré-condições	Estado do sistema antes do bug
Passos para Reproduzir	Sequência detalhada para replicar (numerada)
Resultado Esperado	O que deveria acontecer
Resultado Obtido	O que realmente aconteceu
Evidências	Screenshots, logs do console, vídeos
Observações	Informações adicionais, impacto, workaround

Classificação de Severidade

Severidade	Descrição	Exemplo
Crítica	Sistema quebrado, funcionalidade principal indisponível	Frontend não carrega, erro 500 em todas operações
Alta	Funcionalidade importante não funciona corretamente	Transferência não debita da conta origem
Média	Funcionalidade funciona parcialmente ou com workaround	Mensagem de erro genérica ao invés de específica
Baixa	Problema cosmético ou de usabilidade menor	Botão com cor levemente diferente do especificado

Exemplo Completo de Registro de Bug

ID: BUG-FE-001

Título: Sistema aceita criação de conta com CPF duplicado

Severidade: Alta

Prioridade: Alta

Ambiente:

- Local: <http://localhost:3000> + index.html
- Navegador: Chrome 120.0.6099.129
- Sistema Operacional: Windows 11
- Data: 23/02/2026

Pré-condições:

1. Backend rodando em <http://localhost:3000>
2. Frontend aberto no navegador
3. Já existe uma conta com CPF "111.222.333-44" no sistema

Passos para Reproduzir:

1. Abrir o frontend (index.html) no navegador
2. Localizar o card "Open New Account"
3. Preencher os campos:
 - Full Name: "Teste Duplicado"
 - Email Address: "teste@email.com"
 - Tax ID (CPF): "111.222.333-44" (mesmo CPF de conta existente)
 - Initial Deposit: "500"
4. Clicar no botão "Create Account"
5. Observar a resposta

Resultado Esperado:

- Status HTTP: **400 Bad Request**
- Bloco JSON com **fundo vermelho**
- Mensagem de erro: "CPF já cadastrado" ou similar
- Conta **não deve ser criada**
- Tabela **não deve ser atualizada**

Resultado Obtido:

- Status HTTP: **201 Created**
- Bloco JSON com **fundo verde**
- Conta foi criada com sucesso
- Tabela mostra **duas contas com o mesmo CPF**
- Nenhum erro foi exibido

Evidências:

1. Screenshot da tabela mostrando 2 contas com CPF "111.222.333-44"
2. Screenshot do DevTools → Network tab mostrando resposta 201
3. Screenshot do JSON de resposta verde (sucesso incorreto)

Observações:

- A validação de CPF único **não está funcionando** no backend ou não está sendo aplicada
- Isso viola a regra de negócio: "CPF deve ser único"
- **Impacto:** Múltiplos usuários podem se registrar com o mesmo CPF, causando problemas de identificação e conformidade legal

- **Sugestão:** Implementar validação no backend antes de inserir no banco de dados em memória
-

Exemplo de Bug de Interface

ID: BUG-FE-002

Título: Toast notification não desaparece automaticamente

Severidade: Baixa

Prioridade: Média

Ambiente:

- Local: index.html
- Navegador: Firefox 121
- Sistema Operacional: macOS Sonoma

Pré-condições:

- Frontend aberto e funcionando

Passos para Reproduzir:

1. Criar uma conta com sucesso
2. Observar o toast no canto inferior direito
3. Aguardar mais de 3 segundos

Resultado Esperado:

- Toast aparece com mensagem "Account created successfully!"
- Toast **desaparece automaticamente após 3 segundos**

Resultado Obtido:

- Toast aparece corretamente
- Toast **permanece na tela indefinidamente**
- Não desaparece automaticamente

Evidências:

- Screenshot do toast ainda visível após 10 segundos

Observações:

- Problema cosmético, não impede uso do sistema
- Usuário pode continuar trabalhando normalmente
- Pode ser problema no código JavaScript do setTimeout
- Sugestão: Verificar função showToast() no código

Seu Desafio

Para cada caso de teste que **falhou** durante a execução:

1. Crie um **registro de bug detalhado** seguindo o template
2. Classifique corretamente a **severidade e prioridade**
3. Descreva os **passos para reproduzir** de forma clara (qualquer pessoa deve conseguir replicar)
4. Anexe **evidências** (prints, logs do console, Network tab)
5. Adicione **observações** sobre o impacto no negócio

Dica: Um bom registro de bug economiza tempo do desenvolvedor e acelera a correção.

Nota: Se todos os seus testes **passaram**, parabéns! O sistema está funcionando corretamente. Nesse caso, você pode documentar que nenhum bug foi encontrado durante a execução.

Parte 5: Calcular a Cobertura de Testes

O que é Cobertura de Testes?

Cobertura de testes mede o percentual de requisitos que foram testados em relação ao total de requisitos do sistema.

Fórmula:

$$\text{Cobertura (\%)} = (\text{Requisitos Testados} / \text{Total de Requisitos}) \times 100$$

Como Calcular

Passo 1: Listar todos os requisitos

O Banking Frontend possui **8 requisitos funcionais**:

Requisito	Descrição
RF-FE001	Criar conta
RF-FE002	Listar contas
RF-FE003	Atualizar conta
RF-FE004	Deletar conta
RF-FE005	Realizar depósito
RF-FE006	Realizar transferência

RF-FE007	Feedback visual
RF-FE008	Design responsivo

Total de Requisitos: 8

Passo 2: Verificar quais requisitos foram testados

Revise seus **10 casos de teste** e marque quais requisitos foram cobertos:

Requisito	Casos de Teste	Status
RF-FE001	CT-FE-001, CT-FE-002	Testado
RF-FE002	CT-FE-003	Testado
RF-FE003	CT-FE-004	Testado
RF-FE004	CT-FE-005	Testado
RF-FE005	CT-FE-006, CT-FE-007	Testado
RF-FE006	CT-FE-008, CT-FE-009	Testado
RF-FE007	CT-FE-010	Testado
RF-FE008	-	Não testado

Requisitos Testados: 7

Passo 3: Calcular o percentual

Cobertura = $(7 / 8) \times 100 = 87,5\%$

Cobertura de 87,5% — Excelente! Acima da meta de 80%.

Entregáveis do Exercício

Ao final, você deverá ter criado os seguintes documentos:

1. Casos de Teste

- 10 casos de teste completos seguindo o template
- Cobertura de todos os requisitos (ou justificativa se algum não foi coberto)
- Mix de testes positivos (happy path) e negativos (validações)

2. Plano de Teste

- Escopo definido (o que testar e o que não testar)
- Abordagem e ferramentas documentadas
- Cronograma preenchido com estimativas
- Critérios de aceite claros

3. Relatório de Execução

- Todos os 10 casos executados
- Status registrado (passou/falhou/bloqueado) para cada um
- Observações sobre comportamentos
- Evidências anexadas (screenshots — opcional mas recomendado)

4. Registro de Bugs

- Pelo menos 1 bug registrado (se encontrado)
- Seguindo template completo
- Severidade e prioridade classificadas corretamente
- Passos para reproduzir detalhados
- Evidências anexadas

Nota: Se nenhum bug foi encontrado, documente: "Nenhum bug encontrado durante a execução dos testes. Todos os casos passaram conforme esperado."

5. Matriz de Rastreabilidade

- Requisitos × Casos de Teste × Resultados
- Cálculo de cobertura (%)

Dicas para um Bom Trabalho

Durante a Elaboração dos Casos de Teste

Seja específico: "Criar conta válida" é vago. "Criar conta com saldo inicial 1000 e todos os campos preenchidos corretamente" é melhor.

Use dados reais: CPFs fictícios mas com formato correto (XXX.XXX.XXX-XX).

Teste limites: O que acontece com saldo 0? E com valor muito grande? E com email sem @?

Pense nos erros: Usuários erram. Teste campos vazios, tipos errados, valores negativos.

Teste a UI: Cores corretas? Toast aparece? Tabela atualiza? Botão muda ao passar o mouse?

Durante a Execução

Reinicie quando necessário: Entre um teste e outro, considere recarregar a página ou reiniciar o backend para limpar os dados.

Use as DevTools: F12 é seu melhor amigo. Network tab mostra as requisições, Console mostra erros JavaScript.

Tire prints: Evidências facilitam a comunicação de bugs e comprovam seus testes.

Anote tudo: Se algo estranho acontecer, registre (mesmo que não seja necessariamente um bug).

Teste em ordem lógica: Crie contas primeiro, depois teste operações nelas (depósito, transferência).

No Registro de Bugs

Seja claro e objetivo: Desenvolvedor precisa entender rapidamente o problema.

Um bug por registro: Não misture problemas diferentes em um único registro.

Passos reproduzíveis: Qualquer pessoa deve conseguir replicar seguindo seus passos.

Anexe evidências: Um print vale mais que mil palavras. Screenshot do Network tab é ouro.

Classifique corretamente: Severidade reflete o impacto técnico, Prioridade reflete urgência do negócio.

Na Análise de Cobertura

100% nem sempre é possível: E tá tudo bem. Documente por que algum requisito não foi testado.

Priorize requisitos críticos: Transferência e depósito são mais importantes que cor de botão.

Documente lacunas: Se não testou algo, explique o porquê (ex: "RF-FE008 não testado por limitação de tempo").

Referências e Materiais de Apoio

Documentação do Sistema

- **Especificação Completa:** 3-especificacao-completa.html (fornecido junto com o exercício)
- **Código Frontend:** frontend-web/index.html
- **Backend API:** Swagger em <http://localhost:3000/api-docs> (após subir o servidor)

Perguntas Frequentes

Q: Preciso encontrar bugs para completar o exercício?

A: Não necessariamente. O objetivo é praticar o processo. Se o sistema estiver funcionando corretamente, documente que todos os testes passaram.

Q: Posso criar mais de 10 casos de teste?

A: Sim! 10 é o mínimo recomendado. Quanto mais casos, melhor a cobertura.

Q: E se eu não conseguir subir o backend?

A: Verifique se o Node.js está instalado (node --version), se está na pasta correta (backend/), e se a porta 3000 não está ocupada. Consulte a seção "Como Subir o Sistema".

Q: Posso testar em navegadores diferentes?

A: Sim! Inclusive é uma boa prática. Documente em qual navegador cada teste foi executado.

Q: Como sei se encontrei um bug de verdade?

A: Compare o comportamento obtido com a especificação de requisitos. Se divergir, é um bug. Em caso de dúvida, registre como "possível bug" e descreva o comportamento inesperado.

Q: Preciso anexar screenshots?

A: Não é obrigatório, mas é **altamente recomendado**. Screenshots são evidências valiosas, especialmente para bugs.

Boa sorte e bons testes!